

预训练模型，在逻辑推理方面的能力非常有限，甚至对于 130 亿参数版本的 GPT-3 模型而言，虽然能处理一部分逻辑推理任务，但在复杂度和精确性上仍存在一定局限性。直到 1750 亿参数版本，GPT-3 才能够处理复杂的逻辑推理任务，生成详细和连贯的推理过程。

- .....

这些涌现能力使得大语言模型可以在不进行专项训练的前提下完成各类任务，但同时也带来了诸多挑战，包括模型的可解释性、信息安全与隐私、伦理和公平性问题，以及对计算资源的巨大需求等。解决这些挑战需要在技术、法律和社会层面进行综合考量，以确保大语言模型的健康发展和可持续进步。

## 2.2 大语言模型架构概览

在语言模型的发展历程中，Transformer[42] 框架的问世代表着一个划时代的转折点。其独特的自注意力（Self-Attention）机制极大地提升了模型**对序列数据的处理能力**，在**捕捉长距离依赖关系**方面表现尤为出色。此外，Transformer 框架对**并行计算的支持**极大地加速了模型的训练过程。当前，绝大多数大语言模型均以 Transformer 框架为核心，并进一步演化出了三种经典架构，分别是 Encoder-only 架构，Decoder-only 架构以及 Encoder-Decoder 架构。这三种架构在设计和功能上各有不同。本节将简要介绍这三种架构的设计理念与预训练方式，并分析它们之间的区别以及各自的演变趋势。

### 2.2.1 主流模型架构的类别

本小节将从设计理念、训练方式等角度对 Encoder-only 架构，Decoder-only 架构以及 Encoder-Decoder 架构分别进行简要介绍。

## 1. Encoder-only 架构

Encoder-only 架构仅选取了 Transformer 中的编码器（Encoder）部分，用于接收输入文本并生成与上下文相关的特征。具体来说，Encoder-only 架构包含三个部分，分别是**输入编码**部分，**特征编码**部分以及**任务处理**部分，具体的模型结构如图2.3所示。其中输入编码部分包含**分词**、**向量化**以及**添加位置编码**三个过程。而特征编码部分则是由多个相同的**编码模块**（Encoder Block）堆叠而成，其中每个编码模块包含**自注意力模块**（Self-Attention）和**全连接前馈模块**。任务处理模块是针对任务需求专门设计的模块，其可以由用户针对任务需求自行设计。Encoder-only 架构模型的预训练阶段和推理阶段在输入编码和特征编码部分是一致的，而任务处理部分则需根据任务的不同特性来进行定制化的设计。

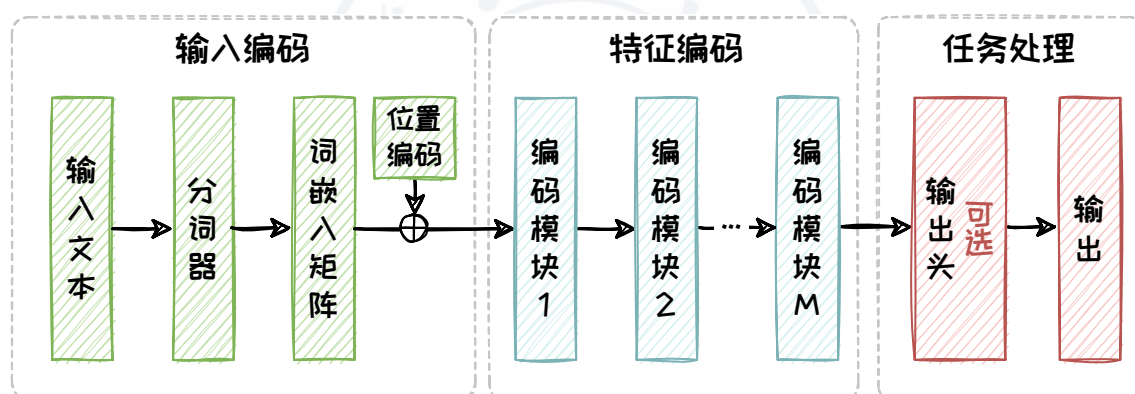


图 2.3: Encoder-only 架构。

在**输入编码**部分，原始输入文本会被分词器（Tokenizer）拆解为 Token 序列，随后通过词表和词嵌入（Embedding）矩阵映射为向量序列，确保文本信息得以数字化表达。具体的过程和细节将在3.1 章节中被具体介绍。接着为了保留文本中单词的顺序信息，每个向量序列会被赋予位置编码（Positional Encoding）。在**特征编码**部分，先前得到的向量序列会依次通过一系列编码模块，这些模块通过自注意力机制和前馈网络进一步提取和深化文本特征。**任务处理**部分在预训练阶段和下游任务适配阶段一般有所差别。在预训练阶段，模型通常使用全连接层作为输出头，

用于完成掩码预测等任务。而在下游任务适配阶段，输出头会根据具体任务需求进行定制。例如，对于情感分析或主题分类等判别任务，只需要添加一个分类器便可直接输出判别结果。但对于文本摘要生成等生成任务，则需要添加一个全连接层，逐个预测后续的 Token。但以这种形式来完成生成任务存在着诸多的限制，例如在每次生成新的 Token 时，都需要重新计算整个输入序列的表示，这增加了计算成本，也可能导致生成的文本缺乏连贯性。本章将在2.3节中对 Encoder-only 架构进行更具体的介绍。

2. Encoder-Decoder 架构

为了弥补 Encoder-only 架构在文本生成任务上的短板，Encoder-Decoder 架构在其基础上引入了一个解码器 (Decoder)，并采用交叉注意力机制来实现编码器与解码器之间的有效交互。

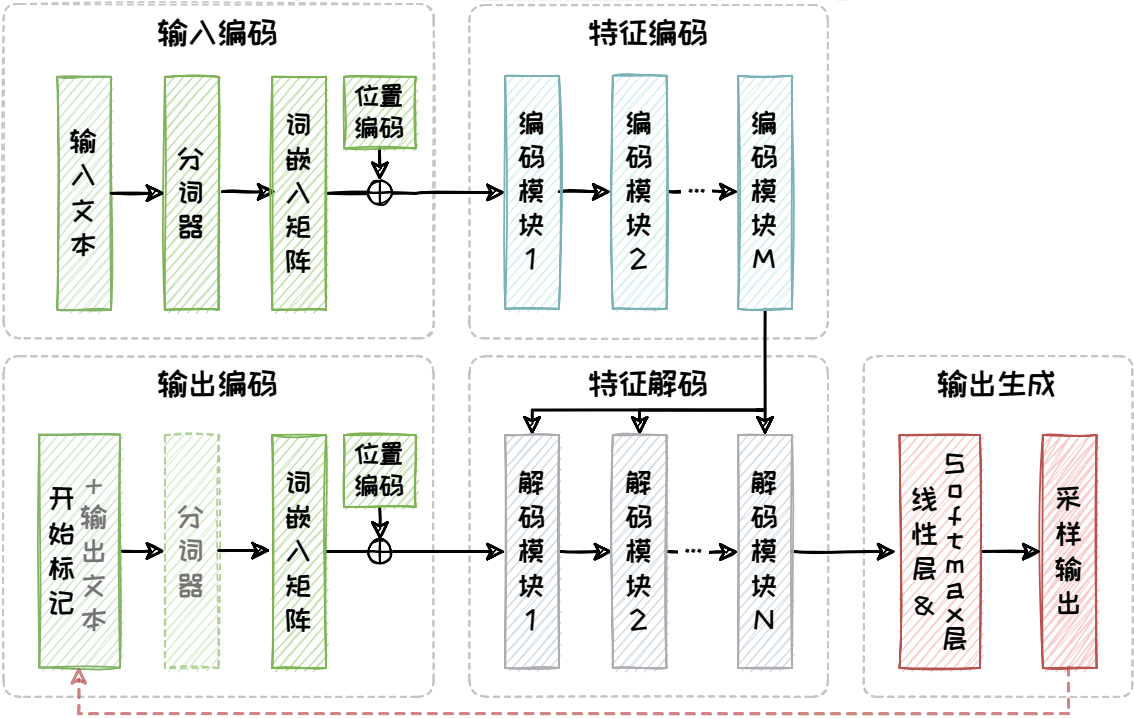


图 2.4: Encoder-Decoder 架构。其中分词器和输出文本只在训练阶段存在，而实现“自回归”的红色虚线只在推理阶段存在。

具体来说，解码器包含了**输出编码**、**特征解码**以及**输出生成**三个部分。其中**输出编码**与编码器中的输入编码结构相同，包含分词、向量化以及添加位置编码三个过程，将原始输入文本转换化为带有位置信息的向量序列。此外，**特征解码**部分与特征编码部分在网络结构上也高度相似，包括掩码自注意力（Masked Self-Attention）模块，交叉注意力模块和全连接前馈模块。其中掩码自注意力模块确保模型只关注上文，不会“预见”未来的信息，从而可以在无“下文泄露”的条件下，进行“自回归”的训练和推理。而交叉注意力模块则负责处理从编码模块向解码模块传递相关信息。**输出生成**部分则由一个线性层以及一个 Softmax 层组成，负责将特征解码后的向量转换为词表上的概率分布，并从这个分布中采样得到最合适的 Token 作为输出。

图2.4展示了 Encoder-Decoder 架构的具体工作流程。在训练阶段，样本中同时包含了输入和真实（Ground Truth）输出文本。其中输入文本首先被输入编码部分转化为向量序列，接着在特征编码模块中被多个堆叠起来的编码模块进一步处理，从而被转化为上下文表示。而输出文本之前会被添加特殊的开始标记 [START]，然后在输出编码部分被分词、词嵌入和位置编码处理后，并行输入到特征解码模块中。接着解码模块使用 Teacher Forcing 技术，在每轮预测时，使用真实输出文本中的已知部分作为输入，并结合从最后一个编码块得到的上下文信息，来预测下一个 Token，计算预测的 Token 和真实 Token 之间的损失，通过反向传播更新模型参数。

在推理阶段，由于缺少了真实的输出文本，所以输出序列原始状态只有开始标记 [START]，也不再需要分词器。模型需要通过自回归的方式，在每轮采样生成 Token 后，会将其拼接到输出序列中，用于下一轮预测。这个过程循环进行，直到生成特定的结束标记 [end] 或达到模型设定的最大输出长度。在这一过程中，由于每轮的输入依赖于上一轮的采样结果，因此只能**一步步地串行输出**。在2.4节中会针对 Encoder-Decoder 架构进行更具体的介绍。

### 3. Decoder-only 架构

为了有效缩减模型的规模以及降低整体的计算复杂度，Decoder-only 架构摒弃了 Encoder-Decoder 架构中的编码器部分以及与编码器交互的交叉注意力模块。在这种架构下，模型仅使用解码器来构建语言模型。这种架构利用“自回归”机制，在给定上文的情况下，生成流畅且连贯的下文。

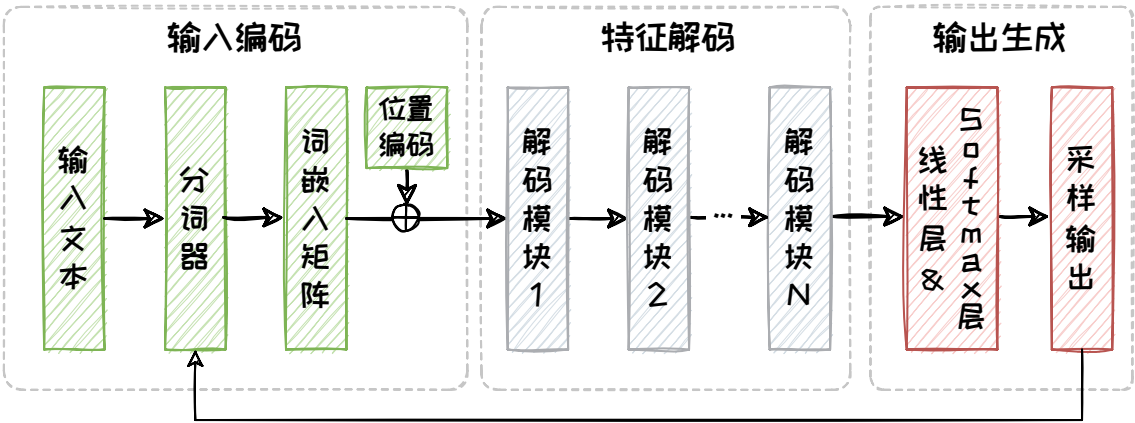


图 2.5: Decoder-only 架构。

Decoder-only 架构同样包含了三个部分，分别是**输入编码**部分、**特征解码**部分以及**输出生成**部分，其具体的模型结构如图2.5所示。Decoder-only 架构的核心特点在于省略了每个编码模块中的交叉注意力子模块，这也是其与传统 Encoder-Decoder 架构中解码器部分的主要区别。在2.5 节中将会对 Decoder-only 架构进行更具体的介绍。

#### 2.2.2 模型架构的功能对比

上述的 Encoder-only、Encoder-Decoder 和 Decoder-only 这三种模型架构虽然都源自于 Transformer 框架，但他们在注意力矩阵上有着显著区别，这也造就了他们在功能以及最终适用任务上的不同。接下来将针对注意力矩阵以及适用任务两个方面对这三种架构的主要区别进行分析。



## 1. 注意力矩阵

注意力矩阵 (Attention Matrix) 是 Transformer 中的核心组件，用于计算输入序列中各个 Token 之间的依赖关系。通过注意力机制，模型可以在处理当前 Token 时，灵活地关注序列中其他 Token 所携带的信息，决定了在这一过程中哪些 Token 能够相互影响。如图2.6所示，三种架构在注意力矩阵上有着显著差异。

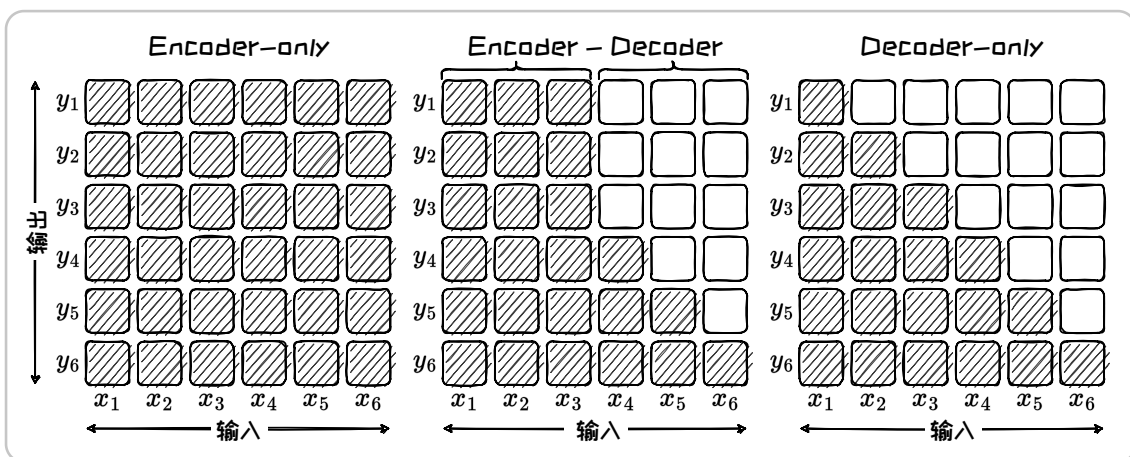


图 2.6: 三种架构的注意力矩阵。

**Encoder-only** 架构中的注意力矩阵来自于自注意力模块，用于捕捉输入序列中各个 Token 之间的关系。Encoder-only 架构的注意力矩阵呈现出“完全”的注意力，即对于每个 Token 的理解都依赖于整个输入序列中的所有 Token。例如，在将输入 Token  $x_i$  转换为上下文向量  $y_i$  的过程中，模型能够综合利用从  $x_1 \sim x_n$  的所有输入信息，这就是所谓的**双向注意力机制**。在这种双向注意力机制的作用下，模型能够同时利用前后文信息，深入理解复杂的语义联系和上下文依赖。

**Encoder-Decoder** 架构中的注意力矩阵较为复杂，它结合了编码器的自注意力、解码器的掩码自注意力以及交叉注意力三种机制。编码器的自注意力矩阵与 Encoder-only 架构类似，用于生成输入序列的全面上下文表示，呈现“完全”的注意力。而解码器的掩码自注意力矩阵则呈现出“下三角”的注意力，确保在生成当前 Token 时，模型只关注之前生成的 Token。此外，交叉注意力机制允许解码器始

终能够动态地参考编码器生成的完整上下文表示，确保输出与输入序列高度相关且连贯。例如，在编码器将输入  $x_i$  转化为上下文向量时，可以利用从  $x_1 \sim x_n$  的所有输入信息；当解码器在生成 Token  $y_i$  的时候，可以参考由  $x_1 \sim x_n$  转化得到的上下文向量以及先前生成的 Token 序列  $y_1 \sim y_{i-1}$  的相关信息。

**Decoder-only 架构**中的注意力矩阵来自于掩码自注意力模块，其特点是呈现出“下三角”的注意力模式。这意味着在预测当前 Token 时，模型只能依赖于已经生成的历史 Token 信息，体现了**单向注意力机制**。例如，在生成 Token  $y_i$  的时候，模型只能考虑先前  $y_1 \sim y_{i-1}$  的信息，这样的设计确保了生成过程的顺序性和文本的连贯性。

### 2. 适用任务

由于各自独特的模型设计以及注意力矩阵上的差异，在同等参数规模下，这三种架构的模型在适用任务上也都有倾向。

**Encoder-only 架构**中的双向注意力机制允许模型在预测每个 Token 时都充分考虑序列中的前后文信息，捕捉丰富的语义和依赖关系。因此，Encoder-only 架构的模型特别适合于**自然语言理解**（Natural Language Understanding, NLU）任务，如情感分析或文本分类等判别任务。然而，由于缺少解码器组件，Encoder-only 架构的模型无法直接生成所需目标序列，因此在自然语言生成任务（Natural Language Generation, NLG）上可能表现不如专门设计的生成模型。

**Encoder-Decode 架构**在 Encoder-only 架构的基础上添加了解码器，使模型能够基于编码器输出的上下文表示逐步生成输出序列。这种编码器和解码器的结合，使得模型可以有效地处理复杂的输入条件，并生成相关且连贯的高质量内容。因此，Encoder-Decoder 架构的模型非常适合于处理各种复杂的**有条件生成任务**，例如机器翻译、文本摘要和问答系统等需要同时理解输入并生成相应输出的场景。但新添加解码器也同样带来了模型规模以及计算量庞大的问题。

**Decoder-only 架构**进一步删除了 Encoder-Decoder 架构中的编码器部分，从而降低了模型本身的计算复杂度。这一架构的模型使用掩码（Mask）操作确保在每个时间步生成当前 Token 时只能访问先前的 Token，并通过自回归生成机制，从起始 Token 开始逐步生成文本。大规模预训练数据的加持使得 Decoder-only 架构的模型能够生成高质量、连贯的文本，在自动故事生成、新闻文章生成此类不依赖于特定的输入文本的**无条件文本生成任务**中表现出色。然而，在模型规模有限的情况下（例如 GPT-1 以及 GPT-2 等模型），由于缺乏编码器提供的双向上下文信息，Decoder-only 架构的模型在理解复杂输入数据时**存在一定局限性**，表现可能不如 Encoder-Decoder 架构。

在不同的历史阶段，三种模型架构分别展现了自身的优势。随着模型规模以及数据规模的显著增长，Decoder-only 架构的模型逐渐占据上风，以其**强大的任务泛化**性能展现出成为“大一统”的架构的潜力。当前，以 GPT-3、GPT-4 等为代表的大型 Decoder-only 语言模型，已经发展出了与人类媲美甚至超越人类的记忆、推理以及执行复杂任务的能力。

### 2.2.3 模型架构的历史演变

随着时间的流逝，我们见证了上述三种架构的演变和流行趋势的更替。在大语言模型的早期发展阶段（2018 年左右），BERT 和 GPT-1 分别作为 Encoder-only 和 Decoder-only 架构的代表几乎同时出现。但受限于当时的模型参数规模，BERT **强大的上下文理解能力**比 GPT-1 **初阶的文本生成能力**更为亮眼。使得 Encoder-only 架构得到了更为广泛的探索和应用，而 Decoder-only 架构吸引得关注则相对关注较少。然而，随着用户对**机器翻译等生成任务需求**的增加，缺乏解码组件的 Encoder-only 架构逐渐无法满足直接生成的需求，因而被逐渐“冷落”。同时，2019 年末诞生了一众 Encoder-Decoder 架构的模型，由于其能够有效处理**序列到序列**（Sequence to



Sequence) 的生成任务, 逐渐成为主流。到了 2019 年末, 随着算力资源的急速发展, 研究者开始寻求不断提升参数量来激发更强的生成能力。得益于其参数易扩展性, Decoder-only 架构下的模型参数量急剧扩充, 文本生成能力大幅提升。自 2021 年之后, 在 GPT-3 等模型的推动下, Decoder-only 架构开始占据主流, 甚至逐渐主导了大语言模型的发展。尽管如此, Encoder-Decoder 架构也仍然活跃在开源社区中, 不断被探索和改进。至于 Encoder-only 架构, 在 BERT 带来最初的爆炸性增长之后, 其关注度有所下降, 但也**仍然在部分判别任务中发挥着重要作用**, 例如文本分类、情感分析、命名实体识别等。总的来讲, 大语言模型的主流架构经历了从 Encoder-only 到 Encoder-Decoder, 再到 Decoder-only 的发展过程, 而引发这种更迭趋势的原因可能是模型本身生成能力以及计算效率上的差异。

**Encoder-only 架构**专注于对输入数据进行深入的表示和理解, 这使得它在理解型任务中表现出色。但当涉及到文本生成任务时, 其**生成能力则不尽如人意**。由于**缺少专门用于生成输出的组件**, 它需要通过迭代进行掩码预测来生成文本, 这需要模型进行多次前向传播以逐步填充文本中的掩码部分, 从而导致计算资源的大量消耗。在当今对人工智能内容生成 (AIGC) 需求不断增长的背景下, 这种架构的应用受到了一定的限制。而 **Encoder-Decoder 架构**使用编码器来处理输入, 并使用解码器来生成输出, 能够处理复杂的序列到序列任务。但相较于后续崛起的 Decoder-only 架构, 其**模型参数规模更为庞大**, 随之带来的是**显著增加的计算复杂度**。特别是在处理长序列数据时, 编码器和解码器均面临**沉重的计算负担**, 这在一定程度上限制了模型的应用效率。相比之下, **Decoder-only 架构**通过仅使用解码器来优化计算流程, 显著简化了模型结构。它采用自回归生成策略, 逐步构建输出文本, 每一步的生成过程仅需考虑已经生成的文本部分, 而不是整个输入序列。这样的设计**减少了模型参数量和计算需求**, 提高了模型本身的可扩展性, 因此非常适合大量文本生成的场景, 在 AIGC 的应用场景下得到了广泛的应用。